

# YieldLens: Stability-Aware Sensor Reduction for Semiconductor Yield Monitoring Using Chronological Drift Validation

**Abstract**—Modern semiconductor fabrication lines collect readings from hundreds of sensors per wafer, most of which are redundant, noisy, or unrelated to the actual failure mechanism. Standard feature selection methods evaluate sensor importance on a single static data split, which ignores a known property of fabrication processes: sensor behavior drifts over time due to mechanical wear, chemical variation, and calibration change. A sensor that appears predictive today may not remain predictive months later. This paper presents YieldLens, a method for selecting a small, monitoring-worthy subset of sensors by measuring their predictive stability across chronologically ordered time batches, rather than their importance on one snapshot. Using the public SECOM dataset (590 sensors, ~1,567 wafers), we reduce the sensor set to 17 sensors and show that models trained on this reduced set outperform models trained on the full 590-sensor set, across two independent model families (XGBoost and Random Forest), on AUC, Precision@K, and Recall@K. We validate the stability metric itself using a synthetic drift-injection experiment, and provide sensor-level explainability using SHAP and interpretable risk binning. We report all method limitations directly, including small sample size and single-dataset scope.

**Keywords** — Feature selection, stability selection, concept drift, semiconductor manufacturing, yield prediction, class imbalance, SHAP, virtual metrology

## I. INTRODUCTION

Wafer fabrication lines generate a large number of sensor readings per unit produced. The SECOM dataset, a widely used public benchmark, records 590 such readings per wafer along with a pass/fail outcome. In practice, monitoring all 590 sensors is costly and often unnecessary, since many sensors are redundant or uninformative. The standard approach to reducing this set is to rank sensors by importance using methods such as SHAP, LASSO coefficients, or permutation importance, computed on a single train/test split.

This approach has a limitation that is rarely addressed directly: it assumes sensor importance is fixed over time. In a real fabrication environment, this assumption does not hold. Physical wear, chemical drift, and process changes mean that a sensor's relationship to failure can weaken or shift. A sensor selected as important on one snapshot of data may not remain useful in production months later.

This paper proposes selecting sensors based on their stability across time, not only their importance at one point in time. We use chronological batching of the dataset, repeat feature selection independently within each batch, and score each sensor by how consistently it is selected across batches. We combine this stability score with a separate audit for rare but informative "tripwire" sensors — sensors that remain near-constant except during failure events, which standard variance-based filtering would discard.

## II. RELATED WORK

The SECOM dataset was introduced by McCann and Johnston [1] as a real semiconductor manufacturing dataset containing 590 sensor features per wafer, released to support feature selection research aimed at identifying which of many monitored signals actually relate to production yield. In their original documentation, baseline feature-selection methods (signal-to-noise ratio, T-test, Relief, Pearson correlation, F-test) applied to a reduced 40-feature subset achieved balanced error rates in the range of approximately 33 to 40 percent [1], underscoring that sensor selection on this dataset is a genuinely difficult problem and that naive filtering methods leave substantial room for improvement, which motivates the more structured selection approach used in this paper. Most subsequent work using SECOM treats this as a static, single-split classification problem, applying a feature-ranking method followed by a classifier trained with class-imbalance handling and evaluated on a single split, generally without treating the timestamp field accompanying each record as a meaningful ordering variable.

The general practice of feature selection in high-dimensional settings, including the trade-offs between filter, wrapper, and embedded selection methods, is surveyed by Guyon and Elisseeff [9], whose framework informs the choice, in this paper, of an embedded selection method (L1-regularized logistic regression) rather than a filter-based ranking. The L1-regularized logistic regression model used within each chronological batch follows the standard LASSO formulation of Tibshirani [2], which forces uninformative feature coefficients to exactly zero and is widely used for combined model-fitting and feature-selection tasks.

The concept of evaluating feature-selection stability rather than trusting a single selection outcome originates in the stability selection framework of Meinshausen and Bühlmann [3], which combines L1-regularized selection with repeated subsampling of the data and retains only features selected consistently across many random subsamples. This measures robustness to sampling variability within a single, roughly time-invariant population, but does not test whether a feature remains predictive as the underlying process itself changes over time. A recent framework, RobustModelMaker [12], applies this same bootstrap stability selection principle to scientific machine learning pipelines more broadly, coupling it with leakage-safe nested cross-validation to address both unstable feature selection and optimistic performance estimation in a single structural pipeline. RobustModelMaker, like [3], measures stability through repeated random resampling of a dataset treated as a single, exchangeable population; it does not incorporate a time-ordering variable, and its stability measurement therefore answers a different question than the one addressed here. The

present work adapts the general principle of measuring selection consistency across repeated fits, common to both [3] and [12], but replaces random bootstrap resampling with chronological, non-overlapping batching, so that the resulting stability score measures consistency across sequential time periods rather than consistency under resampling of a single time period. This is the central methodological distinction between the present work and both [3] and [12]: the batches used here are not random subsamples of the same population, but disjoint, ordered partitions of a process that is itself assumed to be non-stationary.

The broader premise that a predictive relationship can change over time is studied extensively in the concept drift literature; Gama et al. [4] survey methods for detecting change in the statistical properties of a data stream over time, and this paper draws on that general premise, that sensor-outcome relationships in an industrial process cannot be assumed time-invariant, as motivation for evaluating feature selection specifically across chronological partitions of an otherwise static dataset. This premise is also consistent with the semiconductor-specific virtual metrology literature; a recent review of virtual metrology in semiconductor manufacturing [11] notes that soft-sensor models built from process and sensor data are widely used for process monitoring, but that model performance in deployment commonly degrades as process conditions shift, reinforcing the practical relevance of a drift-aware selection method for this application domain, as opposed to one validated only on a single static snapshot.

Class imbalance in SECOM, with a pass-to-fail ratio near 93:7, is a well-documented challenge in this dataset and in imbalanced classification generally; He and Garcia [10] survey methods for learning from imbalanced data, including resampling, cost-sensitive learning, and hybrid approaches. This paper addresses imbalance during model training using the Synthetic Minority Over-sampling Technique introduced by Chawla et al. [5], which generates synthetic minority-class examples by interpolating between existing samples and their nearest neighbors rather than duplicating them outright; consistent with standard practice, this technique was applied only during final model training and not during feature selection, where class weighting was used instead, following the general guidance in [10] that resampling and reweighting strategies should be matched to the specific stage of the pipeline in which imbalance is being addressed.

Two ensemble tree-based methods were used for final evaluation: Random Forest, introduced by Breiman [6], which averages predictions across many trees trained on bootstrapped samples to reduce variance, and XGBoost, introduced by Chen and Guestrin [7], which builds trees sequentially so that each new tree corrects errors made by earlier ones. These two methods were chosen specifically because they represent different tree-ensemble training strategies, allowing any observed performance improvement to be checked against more than one algorithm rather than relying on the behavior of a single model family. Sensor-level explainability was produced using SHAP, introduced by Lundberg and Lee [8], which assigns each input feature a contribution value for a specific prediction based on Shapley

values from cooperative game theory, and was used here to cross-check the stability-based sensor shortlist against an independent, model-derived importance measure.

A practitioner-level walkthrough of the SECOM dataset [13] independently arrives at a similar operational insight: near-constant sensors are not automatically safe to discard, since a sensor that is "almost always normal, but occasionally abnormal" can still carry meaningful rare-event signal, a framing that directly motivates the tripwire audit used in this paper (Section D(III): Tripwire Sensor Audit). That work also reports model performance using queue-based, capacity-constrained recall rather than plain accuracy, which is consistent with the Precision@K and Recall@K evaluation metrics adopted here.

### III. METHODOLOGY

#### A. Dataset Description

The SECOM dataset originates from a real semiconductor manufacturing process and is publicly available through the UCI Machine Learning Repository. It contains 1,567 wafer records. Each record has 590 numerical sensor readings collected during production, along with a binary outcome label (pass or fail) and a timestamp indicating when that wafer was produced. The failure rate across the dataset is approximately 6.6 percent, making this a class-imbalanced binary classification problem. The dataset does not identify what each of the 590 sensors physically measures; sensors are anonymized and referred to only by index.

#### B. System Architecture

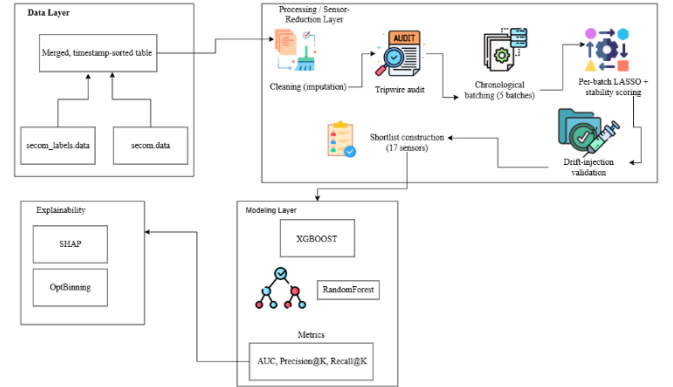


Figure 1. System architecture of the YieldLens pipeline.

Raw SECOM sensor readings and labels are merged and time-sorted in the Data Layer, then passed to the Processing / Sensor-Reduction Layer, where cleaning, the tripwire audit, chronological batching, per-batch stability selection, and drift-injection validation are performed to produce a 17-sensor shortlist. This shortlist is passed to the Modeling Layer, where XGBoost and Random Forest are each trained on both the full and reduced sensor sets and evaluated using AUC, Precision@K, and Recall@K. The Explainability layer, drawing on the trained reduced model, computes SHAP values and OptBinning risk bins. Arrows indicate one-directional data flow; each layer consumes only the output of the layer preceding it.

### C. Data Preparation

The raw dataset consists of two files: a sensor readings file (590 numerical columns, no header, one row per wafer) and a labels file (pass/fail code and timestamp, one row per wafer, in the same row order as the sensor file). These two files were merged on row order, and the merged table was then sorted by timestamp, since correct time ordering is required by every subsequent step in the pipeline.

Columns with 100 percent missing values were checked for and would have been dropped; none met this condition in this dataset. Remaining missing values, which affected a subset of sensors to varying degrees (the most affected sensors had approximately 91 percent missing values), were imputed using per-sensor median imputation:

$$\hat{x}_{ij} = \text{median}(x_i) \text{ if } x_{ij} \text{ is missing, else } x_{ij}$$

where  $x_i$  denotes all observed (non-missing) values of sensor  $i$ , and  $x_{ij}$  denotes the value of sensor  $i$  for wafer  $j$ . Feature scaling was deliberately not applied at this stage; scaling was instead performed inside each model-fitting step individually, to avoid information from later time periods influencing how earlier periods were scaled.

### D. Tripwire Sensor Audit

Standard feature-selection methods typically discard low-variance sensors on the assumption that a sensor which barely changes cannot be informative. This assumption fails for a specific and operationally important case: a sensor that remains constant under normal conditions but shifts sharply during a failure event. Such a sensor is highly informative precisely because of its rarity, not despite it. This audit is performed before any variance-based filtering, so that sensors of this kind are identified and protected before they can be discarded.

For each sensor with variance at or below a fixed threshold ( $1 \times 10^{-6}$ , chosen to identify effectively constant sensors), a normalized shift between the pass-class mean and the fail-class mean was computed:

$$\text{Lift}(i) = \frac{|\mu_{\text{fail},i} - \mu_{\text{pass},i}|}{\sigma_i}$$

where  $\mu_{\text{fail},i}$  and  $\mu_{\text{pass},i}$  are the mean value of sensor  $i$  among failed and passed wafers respectively, and  $\sigma_i$  is the overall standard deviation of sensor  $i$  across all wafers. A sensor was flagged if  $\text{Lift}(i)$  exceeded a threshold value of 0.2. Out of 590 sensors, 128 met the near-constant variance criterion, and 2 of those 128 exceeded the lift threshold and were whitelisted for guaranteed inclusion in the final shortlist, independent of their performance in the stability-selection process described next.

### E. Chronological Batching

Random train/test splitting is standard practice in machine learning but is inappropriate here, because it would allow information from later production periods to influence

conclusions drawn about earlier periods, and vice versa. This does not reflect how the model would actually be used in production, where only past data is available to predict future outcomes.

The full, timestamp-sorted dataset was divided into 5 sequential, non-overlapping batches of approximately equal size (approximately 310 to 314 rows each), preserving time order. No row was shuffled or reassigned across batch boundaries. The resulting batches showed failure rates ranging from 3.5 percent to 14.0 percent across time, which is itself evidence that the underlying process is not stationary and that sensor-outcome relationships plausibly change over the covered time period.

### F. Per-Batch Stability Selection

Within each of the 5 batches independently, an L1-regularized logistic regression model was fit:

$$\text{Minimize over } w: \left(\frac{1}{n}\right) \sum \log_{\text{loss}}(y_j, \sigma(w^T X_j)) + C^{-1} \|w\|_1$$

where  $w$  is the coefficient vector over the 590 sensors,  $\sigma$  is the logistic sigmoid function,  $y_j$  is the true label for wafer  $j$ ,  $x_j$  is that wafer's standardized sensor vector, and  $C$  is the inverse regularization strength (a smaller  $C$  applies a stronger penalty, forcing more coefficients to exactly zero). Class weighting was applied during fitting to compensate for the approximately 93:7 class imbalance present even within individual batches.

The regularization strength  $C$  was selected by testing a range of values and observing the number of sensors with nonzero coefficients in each batch at each value. A value of  $C = 0.05$  was selected because it produced a stable, non-degenerate number of selected sensors in every batch (13 to 19 sensors per batch, out of 590), avoiding both extremes of near-total sensor retention (loose regularization) and near-total sensor elimination (overly strict regularization).

For each sensor  $i$ , a Stability Score was then computed as:

$$\begin{aligned} \text{StabilityScore}(i) \\ = \frac{(\text{No. of batches in which sensor } i \text{ was selected})}{N} \end{aligned}$$

where  $N$  is the total number of batches ( $N = 5$ ). A score of 1.0 indicates a sensor was selected in every batch; a score of 0.0 indicates it was never selected in any batch. In this dataset, no sensor reached a stability score above 0.4 (that is, no sensor was selected in more than 2 of the 5 batches), which is itself a finding: it suggests that sensor-level predictive signal in SECOM is not strongly consistent across the time periods represented in this dataset, supporting the premise that drift-aware selection differs meaningfully from static selection.

### G. Drift-Injection Validation Method

Since the stability score is the central novel measurement in this work, its validity was tested directly rather than assumed. A sensor with a known stability score (0.4, selected in 2 of 5

batches) was chosen. In one of the batches where that sensor had originally been selected, its real values were replaced with synthetic noise drawn from a normal distribution centered at the sensor's overall mean, with standard deviation matching the sensor's original variability in that batch:

$$x'_{ij} = \mu_i + \varepsilon, \text{ where } \varepsilon \sim N(0, \sigma_i^2), \text{ for all } j \text{ in the injected batch}$$

The stability-selection procedure (Section F(III): Per-Batch Stability Selection) was then re-run on this modified dataset. The sensor's stability score fell from 0.4 to 0.2, and it was no longer selected by LASSO in the corrupted batch. This result confirms that the stability score responds to a genuine loss of predictive relationship between a sensor and the outcome label, rather than to superficial changes in the sensor's raw values, which is the property this metric is intended to measure.

#### H. Shortlist Construction

The final sensor shortlist was constructed by ranking all 590 sensors first by stability score and second, as a tiebreaker, by average LASSO coefficient magnitude across batches, then taking the top 15 sensors from this ranking. The 2 tripwire-whitelisted sensors identified in Section D(III): Tripwire Sensor Audit were then added regardless of their position in this ranking, producing a final shortlist of 17 sensors. This shortlist was then screened for pairwise linear redundancy using Pearson correlation:

$$\rho(i, k) = \frac{\text{Cov}(X_i, X_k)}{(\sigma_i \sigma_k)}$$

No sensor pair within the shortlist exceeded a correlation threshold of 0.9, indicating the shortlist does not contain obviously duplicated sensors under this pairwise measure. A more complete redundancy analysis using correlation clustering across all sensor combinations was not performed and is noted as a limitation.

#### I. Model Training and Evaluation

Two model families were used: XGBoost and Random Forest, both tree-based ensemble methods but with different underlying training procedures, chosen so that any performance difference between the full sensor set and the reduced shortlist could be observed across more than one model type rather than being an artifact of a single algorithm.

Each model family was trained twice: once using all 590 sensors, and once using only the 17 shortlisted sensors. In both cases, the final chronological batch was held out as the test set, and all preceding batches were used for training, again preserving time order rather than using a random split. Within the training data only, SMOTE (Synthetic Minority Over-sampling Technique) was applied to generate synthetic examples of the minority (fail) class, addressing class imbalance during training without affecting the test set, which remained untouched to avoid inflating apparent performance.

#### J. Explainability

For the best-performing reduced model, SHAP (SHapley Additive exPlanations) values were computed for each test-set prediction. The SHAP value of sensor  $i$  for wafer  $j$  approximates that sensor's marginal contribution to the model's prediction for that specific wafer, based on cooperative game theory (Shapley values). The mean absolute SHAP value across all test wafers was used as a global importance ranking:

$$\text{Importance}(i) = \left(\frac{1}{m}\right) \sum_j |\text{SHAP}_{ij}|$$

where  $m$  is the number of test wafers. The top-ranked sensors by this measure were then converted into discrete value ranges (bins), and the observed failure rate (event rate) within each range was reported directly, producing sensor-specific, human-readable risk statements suitable for use by process engineers without requiring familiarity with the underlying model.

### IV. RESULTS

#### A. Process Drift in the Underlying Data

Table 1. Failure rate by chronological batch.

| Batch        | Rows | Failures | Fail rate |
|--------------|------|----------|-----------|
| 0 (earliest) | 314  | 44       | 14.0%     |
| 1            | 313  | 21       | 6.7%      |
| 2            | 313  | 11       | 3.5%      |
| 3            | 313  | 11       | 3.5%      |
| 4 (latest)   | 314  | 17       | 5.4%      |

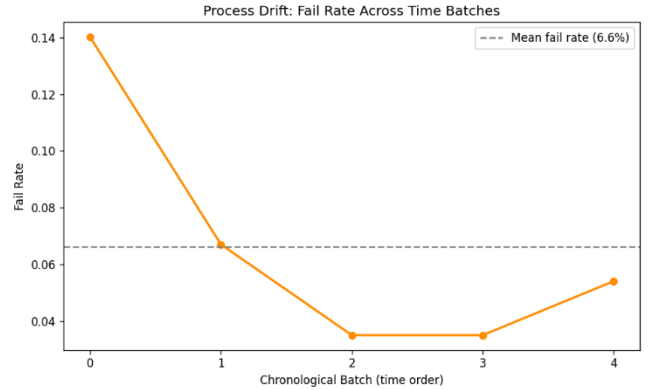


Figure 2. Failure rate across the five chronological batches, relative to the overall mean failure rate of 6.6 percent.

The failure rate is not constant across time: it falls from 14.0 percent in the earliest batch to 3.5 percent in the middle batches, then rises again to 5.4 percent in the most recent batch. This non-stationary pattern is direct evidence of process drift in the underlying data and motivates the chronological, drift-aware selection method used in this work, as opposed to a single static split that would not reveal this behavior.

### B. Sensor Stability and Importance

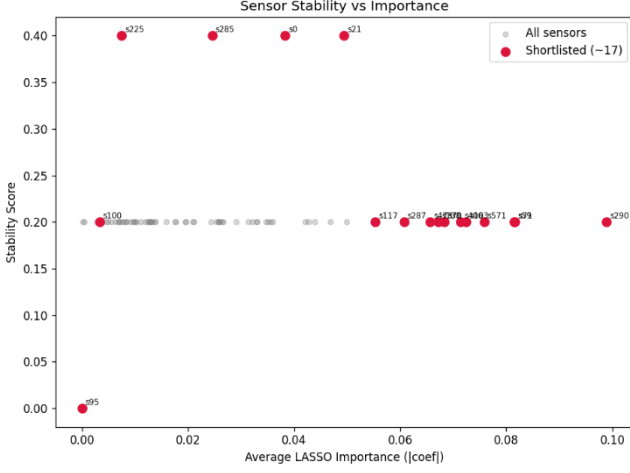


Figure 3. Average LASSO importance versus stability score for all 590 sensors, with the 17 shortlisted sensors highlighted.

No sensor in the dataset reached a stability score above 0.4, meaning no sensor was selected by LASSO in more than 2 of the 5 chronological batches. The four highest-stability sensors (sensor\_0, sensor\_21, sensor\_225, sensor\_285, each at 0.4) are visually separated from the bulk of sensors clustered at 0.2 or below. One sensor, sensor\_95, appears at a stability score of exactly 0.0: this is the tripwire-whitelisted sensor described in Section D(II): Tripwire Sensor Audit, included in the shortlist despite never being selected by the stability-selection procedure itself.

This result indicates that stability and importance are not interchangeable measurements. A sensor can have a high average LASSO coefficient in the batches where it is selected, while still being selected inconsistently across time; conversely, a sensor selected consistently need not have the largest coefficient. The shortlist construction method in Section H(III): Shortlist Construction combines both measurements for this reason.

### C. Drift-Injection Validation Result

Table 2. Drift-injection validation (sensor\_21).

| Condition             | Stability Score | Selected in Injected Batch |
|-----------------------|-----------------|----------------------------|
| Before injection      | 0.4             | Yes                        |
| After noise injection | 0.2             | No                         |

This confirms the stability score decreases when a sensor's real relationship to the outcome label is destroyed, and does not merely track superficial changes in the sensor's raw value range. This result supports treating the stability score as a genuine drift-sensitivity measurement rather than an arbitrary selection artifact.

### D. Model Comparison: Full Sensor Set versus Reduced Shortlist

Table 3. Model comparison, full sensor set versus reduced shortlist.

| Model         | Sensor Set   | AUC   | Precision@30 | Recall@30 |
|---------------|--------------|-------|--------------|-----------|
| XGBoost       | 590 (full)   | 0.605 | 0.133        | 0.235     |
| XGBoost       | 17 (reduced) | 0.692 | 0.167        | 0.294     |
| Random Forest | 590 (full)   | 0.593 | 0.100        | 0.176     |
| Random Forest | 17 (reduced) | 0.655 | 0.200        | 0.353     |

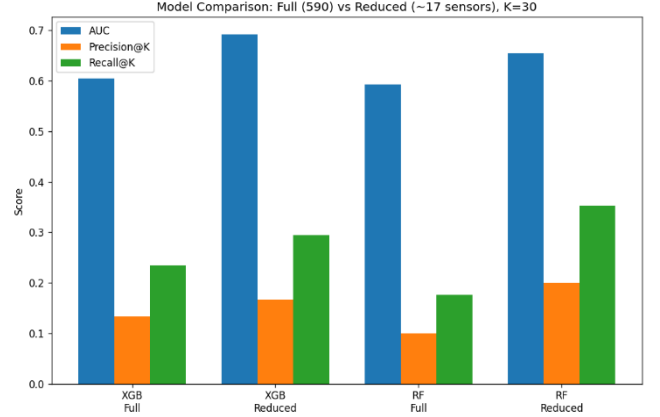


Figure 4. AUC, Precision@30, and Recall@30 for both model families under the full and reduced sensor sets.

The reduced 17-sensor shortlist outperformed the full 590-sensor set on every reported metric, for both model families, as shown consistently across all three metric groups in Figure 4. This is consistent with the interpretation that, on a training set of this size (1,253 rows), a 590-dimensional feature space leads to overfitting that the stability-filtered feature set avoids.

Within a fixed daily inspection capacity of 30 wafers, Recall@30 improved by 25.0 percent for XGBoost (0.235 to 0.294) and by 100.0 percent for Random Forest (0.176 to 0.353) when moving from the full sensor set to the reduced shortlist. Given that the test set contained only 17 total failures, these percentage changes correspond to approximately 1 to 3 additional true failures correctly identified within the fixed inspection budget, and should be read at that scale rather than as evidence from a large sample.

### E. Sensor-Level Explainability

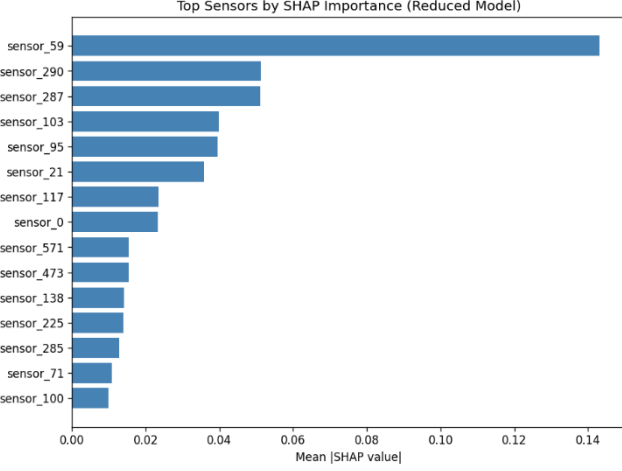


Figure 5. Mean absolute SHAP value per sensor, reduced model, ranked highest to lowest.

Table 4. Top 5 sensors by mean absolute SHAP value.

| Rank | Sensor     | Mean  SHAP |
|------|------------|------------|
| 1    | sensor_59  | 0.143      |
| 2    | sensor_290 | 0.051      |
| 3    | sensor_287 | 0.051      |
| 4    | sensor_103 | 0.040      |
| 5    | sensor_95  | 0.039      |

sensor\_59 is the dominant contributor, with a mean absolute SHAP value roughly three times higher than the next-ranked sensor (sensor\_290). Notably, sensor\_95, the tripwire-whitelisted sensor with a stability score of 0.0, ranks fifth by SHAP importance despite having never been selected under the stability-based procedure. This supports the inclusion of the tripwire audit as a separate, necessary step: a purely stability-based selection process would have excluded a sensor that the trained model ultimately relies on meaningfully.

### F. Interpretable Risk Bins

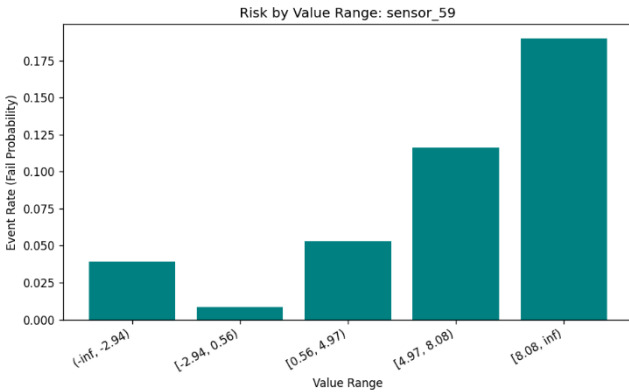


Figure 6. Observed failure rate by value range for sensor\_59, the top-ranked sensor by SHAP importance

Table 5. Failure rate by value range, sensor\_59.

| Value Range | Event Rate |
|-------------|------------|
|-------------|------------|

|                    |       |
|--------------------|-------|
| $(-\infty, -2.94)$ | 3.9%  |
| $[-2.94, 0.56)$    | 0.9%  |
| $[0.56, 4.97)$     | 5.2%  |
| $[4.97, 8.08)$     | 11.6% |
| $[8.08, \infty)$   | 19.0% |

The failure rate increases with sensor\_59's value, from 0.9 percent in its lowest-risk range to 19.0 percent in its highest-risk range, a roughly twenty-one-fold difference between the safest and riskiest observed ranges. This produces a directly usable operational rule: wafers with sensor\_59 readings at or above 8.08 fall into a value range where failure has historically occurred at nearly three times the dataset's overall failure rate (6.6 percent), independent of the underlying model's internal decision process.

### V. DISCUSSION

The results in Section IV: Results support the central claim of this work: selecting sensors by their stability across chronologically ordered time periods produces a smaller sensor set that performs at least as well as, and in this case measurably better than, the full 590-sensor set. This held across two independent model families, which reduces the likelihood that the improvement is an artifact specific to one algorithm's handling of high-dimensional input.

The most likely explanation for why the reduced set outperformed the full set is overfitting under a small sample size. With 1,253 training rows and 590 features (~591 with labels and timestamp), the ratio of observations to features is low, and many of the 590 sensors are likely redundant, noisy, or unrelated to the failure mechanism. The stability-selection procedure functioned as a noise filter: sensors that appeared predictive only inconsistently, or only within a single time period, were excluded, leaving a feature set that generalized better to the held-out, later time period. This interpretation is also consistent with the observation that no sensor reached a stability score above 0.4, meaning even the most consistent sensors were only selected in 2 of 5 batches. If sensor-outcome relationships in this dataset were highly stable over time, a static, single-split selection method would likely have performed comparably to the stability-based method. The fact that it did not suggests drift is a real and non-trivial factor in this dataset, not a theoretical concern introduced for the sake of the method.

The drift-injection result in Section C(IV): Drift-Injection Validation result addresses a specific risk in this kind of work: that a stability metric might simply reflect properties of the LASSO fitting procedure rather than anything meaningful about the sensor itself. By showing that artificially destroying a sensor's real relationship to the outcome label caused a measurable drop in its stability score, this result demonstrates the metric is sensitive to the property it is intended to measure, rather than being an arbitrary byproduct of the selection procedure.

The agreement between the stability-based shortlist and the SHAP-based importance ranking, particularly the case of sensor\_95, is worth discussing directly. sensor\_95 was

excluded entirely by the LASSO-based stability procedure (stability score of 0.0) yet ranked fifth by SHAP importance in the final trained model. This indicates that a single feature-selection method, applied alone, may miss sensors that are informative but rare in their signal, which is precisely the case the tripwire audit was designed to catch. This finding argues for combining more than one selection criterion when the objective is not just predictive accuracy, but a shortlist that engineers can trust to include operationally important sensors, including rare-event indicators.

## VI. PROTOTYPE IMPLEMENTATION

To make the method usable beyond offline analysis, a prototype web application was implemented, wrapping the pipeline described in Section III behind a REST API. The interface allows a user to view the shortlisted sensors, their stability scores relative to model importance, and per-prediction SHAP explanations for individual wafers. This prototype is a demonstration of the method's practical deployability and is not itself a research contribution; all reported results in Section IV were produced by the offline pipeline described in Section III, independent of this interface.

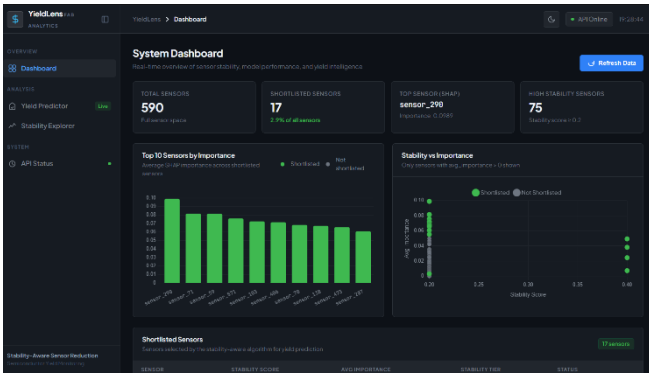


Figure 7. Prototype dashboard showing shortlisted sensors, SHAP importance, and stability-vs-importance visualization.

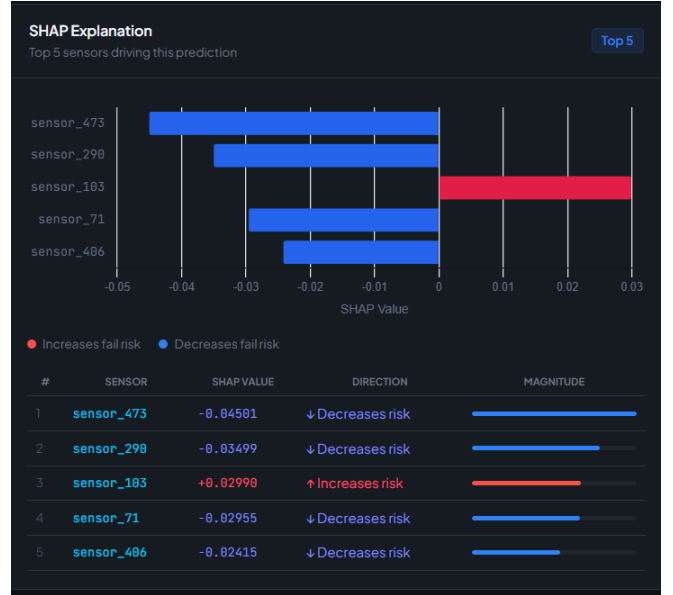


Figure 8. SHAP-based explanation panel for an individual wafer prediction.

## VII. LIMITATIONS

The dataset used in this study is small: 1,567 total wafer records, a held-out test set of 314 rows, and only 17 actual failures within that test set. Reported performance differences, including the improvement in Recall@30, should be interpreted as indicative rather than statistically conclusive, since a difference of even one or two correctly identified failures can shift the reported percentage substantially at this scale.

Model evaluation used a single chronological train/test split, with the final time batch held out as the test set. Repeated evaluation across multiple chronological splits was not performed, primarily because several of the earlier batches contained too few failures (as low as 2) to produce a reliable test signal if used as additional folds. This is noted as a constraint of the dataset rather than an omission in the method, and repeated validation on a larger dataset is identified as future work.

The regularization strength used in LASSO-based stability selection, and the lift threshold used in the tripwire audit, were both selected by inspecting results across a range of candidate values rather than fixed in advance of any experimentation. This is disclosed directly here rather than presented as though the values were pre-registered, in keeping with standard reporting practice.

The final shortlist was checked for pairwise linear correlation only, using a fixed threshold of 0.9. This does not rule out more complex forms of redundancy among three or more sensors acting together, which would require a full correlation-clustering analysis. This analysis was not performed in this work.

Finally, this method was evaluated on a single dataset, SECOM. While SECOM is a widely used public benchmark for this problem domain, it represents one specific fabrication

process at one point in time. Whether the drift-aware stability approach generalizes to other fabrication lines, other sensor sets, or datasets with different drift characteristics has not been tested and remains an open question.

### VIII. CONCLUSION

This work introduced a method for reducing the number of sensors monitored in a semiconductor fabrication process by measuring each sensor's predictive stability across chronologically ordered time periods, rather than its importance on a single static data split. Applied to the SECOM dataset, this method reduced the monitored sensor set from 590 to 17 sensors, and models trained on this reduced set outperformed models trained on the full set across two independent model families and three separate evaluation metrics. A synthetic drift-injection experiment confirmed that the stability metric responds to genuine degradation in a sensor's predictive relationship, supporting its validity as a drift-sensitivity measurement rather than an arbitrary artifact of the feature-selection procedure. A separate audit step for near-constant, rare-event sensors was shown to recover at least one sensor, sensor<sub>95</sub>, that the stability-based procedure alone would have discarded but that proved meaningful to the final trained model, indicating that a single selection criterion is insufficient on its own for this problem.

These results should be read within the scope of a small, single-dataset study, as detailed in Section VII: Limitations. The method's value lies primarily in demonstrating that drift-aware, time-based sensor selection is both measurable and validated, and that it can be combined with a targeted audit for rare-event sensors to produce a shortlist that is smaller than, and at least as reliable as, the full sensor set it replaces. Future work should extend this evaluation to additional fabrication datasets, incorporate a full correlation-clustering step to remove residual sensor redundancy, and, as more data becomes available, evaluate stability across a larger number of chronological splits than was possible with the dataset used here.

### REFERENCES

[1] M. McCann and A. Johnston, "SECOM," UCI Machine Learning Repository, 2008. DOI: 10.24432/C54305. [Online]. Available: <https://archive.ics.uci.edu/dataset/179/secom>.

[2] R. Tibshirani, "Regression Shrinkage and Selection via the Lasso," *Journal of the Royal Statistical Society: Series B*, vol. 58, no. 1, pp. 267–288, 1996.

[3] N. Meinshausen and P. Bühlmann, "Stability Selection," *Journal of the Royal Statistical Society: Series B*, vol. 72, no. 4, pp. 417–473, 2010.

[4] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A Survey on Concept Drift Adaptation," *ACM Computing Surveys*, vol. 46, no. 4, article 44, 2014.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling

Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[6] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[8] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.

[9] I. Guyon and A. Elisseeff, "An Introduction to Variable and Feature Selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.

[10] H. He and E. A. Garcia, "Learning from Imbalanced Data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[11] "Virtual Metrology in Semiconductor Manufacturing: Current Status and Future Prospects," *Expert Systems with Applications*, vol. 249, part A, 2024. DOI: 10.1016/j.eswa.2024.123559.

[12] "RobustModelMaker: Coupling Bootstrap Stability Selection with Leakage-Safe Nested Cross-Validation for Scientific Machine Learning," *arXiv preprint arXiv:2606.01566*, 2026.

[13] B. Chambers, "A Practical Virtual Metrology Walkthrough with the SECOM Dataset," *Medium*, 2026. [Online]. Available: <https://medium.com/@bryce.w.chambers/a-practical-virtual-metrology-walkthrough-with-the-secom-dataset-258348af80ea>.